

Lab 3: High-Level Synthesis

In this lab, you explore High-Level Synthesis (HLS) optimizations. Your main objective is to apply techniques discussed in the lectures to minimize the total number of cycles required to execute a specific task.

1 Available Optimizations for Static HLS

The HLS syntax and pragmas used in this lab have been adapted from the [Vitis HLS documentation \(2025.2\)](#). Feel free to check it to get further explanation and examples of utilizations.

pipeline: overlaps iterations inside one loop execution.

1.1 Pipelining

rewind: overlaps the end of one loop execution with the start of the next loop execution.

Rather than waiting for a loop iteration to finish entirely before the next one begins, pipeline allows overlapping multiple executions concurrently. A pipelined function can process new inputs every N cycles, where N is the II of the loop. An II of 1 processes a new input every clock cycle.

You can instruct the compiler to implement pipelining, using:

```
#pragma HLS pipeline II=<int> rewind style=<stp|frp|flp>
```

Where:

II: Specifies the desired initiation interval for the pipeline. In general, choose II by identifying the bottleneck per iteration (memory ports, operator availability, or loop dependencies), then set II to the smallest feasible value that satisfies all of them. Make sure to enter a valid value!

rewind: (Optional) Enables continuous execution across loop boundaries. Normally, a pipelined loop must flush its epilogue before starting the prologue of its next execution. This option overlaps these phases, eliminating bubbles between consecutive executions of the loop. Note that it requires the loop to have a single entry and a single exit in the control flow, i.e., one path into the loop body and one path out of it (no early exits such as `break`/`return` from inside the loop).

style: Specifies the type of pipeline to use.

- **stp:** Stall pipeline. Runs only when input data is available otherwise it stalls.
- **frp:** Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power as the pipeline registers are clocked even if there is no data.
- **flp:** Flushable pipeline. Similar to `stp`, except the pipeline stops reading new inputs but finishes executing ongoing operations. This type of pipeline typically consumes more resources and/or can have a larger II because resources cannot be shared among pipeline iterations.

array_partition factor=N = split one array into N banks

Each bank has its own port = more parallel memory access

More banks can reduce II and make pipeline faster

But cost: more BRAM/registers + more hardware + more power/routing

1.2 Array Partitioning

In standard software, an array is a contiguous block of memory. However, in HLS on FPGAs, arrays are typically synthesized into Block RAMs (BRAMs). In this class, assume BRAMs only have a single memory port, meaning they can perform a single read or write in a single cycle per array. Because of that constraint, without enough memory ports the hardware is forced to stall the execution, which prevents it from achieving an ideal II .

To solve this bandwidth limitation, you can instruct the compiler to divide the array into smaller separate memory banks, using:

How is HLS II actually calculated?

You can use the following process:

Step 1: Start by trying $II = 1$.



Step 2: Check whether the memory ports are sufficient.

Are there too many accesses to the same array/bank in the same cycle?

Step 3: Check whether the operator resources are sufficient.

Are multipliers or adders overused in the same cycle?

Step 4: Check loop-carried dependencies.

Is a value produced by one iteration ready before a later iteration needs it?

Step 5: If the schedule is not valid, increase II ,
add more hardware resources, or rewrite the code.

So intuitively, the final II is constrained by:

$II = \max(\text{memory-port lower bound},$
 $\text{operator-resource lower bound},$
 $\text{loop-carried-dependency lower bound})$



However, this is only an intuitive rule. The actual II also depends on the detailed schedule: where each load, store, multiplication, and addition is placed in the pipeline.

The clearest way to understand **Loop A / Loop B** is that they are actually the **same inner loop** executed **multiple times** by an outer loop.

C code example

```
</> C
void example(int x[2][4], int y[2][4]) {
    for (int t = 0; t < 2; t++) {
        // This inner loop is the pipelined loop
        for (int i = 0; i < 4; i++) {
#pragma HLS pipeline II=1
            y[t][i] = x[t][i] * 2 + 1;
        }
    }
}
```

Here:

```
t = 0 inner loop = Loop execution A
t = 1 inner loop = Loop execution B
```

So:

```
Loop A: t = 0, i = 0, 1, 2, 3
Loop B: t = 1, i = 0, 1, 2, 3
```

Without `rewind`

```
<> C
void example_no_rewind(int x[2][4], int y[2][4]) {

    for (int t = 0; t < 2; t++) {

        for (int i = 0; i < 4; i++) {
#pragma HLS pipeline II=1
            y[t][i] = x[t][i] * 2 + 1;
        }
    }
}
```

Conceptually, execution looks like this:

```
Loop A: t = 0
i=0 → i=1 → i=2 → i=3 → drain pipeline

bubble bubble bubble

Loop B: t = 1
i=0 → i=1 → i=2 → i=3 → drain pipeline
```

That means:

```
After A's i=3 enters the pipeline,
the hardware waits until all unfinished operations from Loop A are completed.
Only then can B's i=0 enter the pipeline.
```

So there are empty cycles between the two loop executions.

With `rewind`

```
</> C
void example_with_rewind(int x[2][4], int y[2][4]) {
    for (int t = 0; t < 2; t++) {
        for (int i = 0; i < 4; i++) {
            #pragma HLS pipeline II=1 rewind
            y[t][i] = x[t][i] * 2 + 1;
        }
    }
}
```

Conceptually, execution looks like this:

```
Loop A: t = 0
i=0 → i=1 → i=2 → i=3
      ↘
Loop B:          i=0 → i=1 → i=2 → i=3
```

That means:

A's i=2 / i=3 may still be in the later pipeline stages,
while B's i=0 can already enter the earlier pipeline stages.

So:

Loop A's epilogue
and
Loop B's prologue

can overlap.

This matches the Lab 3 explanation: `rewind` allows a pipelined loop to execute continuously across loop boundaries, instead of flushing the epilogue before starting the next prologue. [Lab_3_Solutions](#)

Timing example

Assume:

pipeline latency = 4 cycles
II = 1

Without **rewind**

Cycle:	0	1	2	3	4	5	6	7	8	9	10
Start:	A0	A1	A2	A3	-	-	-	B0	B1	B2	B3

The - cycles are bubbles because Loop A is still draining.

With **rewind**

Cycle:	0	1	2	3	4	5	6	7
Start:	A0	A1	A2	A3	B0	B1	B2	B3

There are no bubbles between Loop A and Loop B.

```
#pragma HLS array_partition variable=<str> type=<cyclic|block|complete> \
                                factor=<int> dim=<int>
```

Where:

variable: Specifies the name of the array to be partitioned.

type: Determines how the array elements are distributed among the new memory banks. The available types are:

- **cyclic:** Interleaves cyclically the array elements across the memory banks. For example, if `factor=3` is used on an array:
 - Element 0 is assigned to the first new array.
 - Element 1 is assigned to the second new array.
 - Element 2 is assigned to the third new array.
 - Element 3 is assigned to the first new array again.
 - Etc.
- **block:** Creates N smaller arrays that are consecutive, where N is defined by the factor.
- **complete:** Fully decomposes the array into individual registers instead of BRAMs. This guarantees concurrent access to every element in a single cycle, but requires significantly more logic resources.

factor: Defines the number of smaller arrays (memory banks) to create.

dim: Specifies which dimension of a multi-dimensional array to partition (use 0 to partition all dimensions).

1.3 Loop Unrolling

You can unroll loops to create multiple independent operations rather than a single collection of operations. The `UNROLL` pragma transforms loops by creating multiples copies of the loop body in the RTL design, which allows some or all loop iterations to occur in parallel. If the loop is partially unrolled, the loop bounds does not need to be known at compile time and the compiler automatically adds an exit check to ensure the functionality remains identical to the original code.

Instruct the compiler using the following syntax:

```
#pragma HLS unroll factor=<int> skip_exit_check
```

Where:

factor: Specifies how many times the loop body is unrolled. If not specified, the loop is fully unrolled.

skip_exit_check: (Optional) Applies only if partial unrolling is specified. Instructs the compiler to omit the hardware logic that checks the exit condition between unrolled iterations. Removing this check saves logic resources and simplifies control logic, but it will cause incorrect behavior if the iteration count is not a multiple of the factor.

2 A Concrete Example on Static HLS

Consider the following C kernel:

```
1 void simpleDesign(int a[1024], int b[2048], int o[1024]) {
2     for(int i = 0; i < 1024; ++i){
3         o[i] = a[i] + b[2*i] * b[2*i+1];
4     }
5 }
```

Answer the questions assuming the following fixed latencies:

Operation	Memory Load/Store	Addition	Multiplication
Number of Cycles	1	1	4

Additionally, assume every array maps to a single BRAM, meaning each array has a single memory port. Address calculations (e.g., $2 \cdot i$ and $2 \cdot i + 1$), as well as any operations occurring before & after the loop (such as variable initialization), have a negligible cost and can be ignored in the questions.

For each question, write the modification on the code in order to apply the requested optimization. Each optimization is cumulative, i.e., it considers all previous ones to be implemented. Fill out the scheduling template for the first 3 iterations, starting at $i = 0$. Write the iteration count (i.e., the current i) on the vertical axis. In addition, clearly indicate the latency induced by your schedule, the II , along with the number of resources that need to be instantiated.

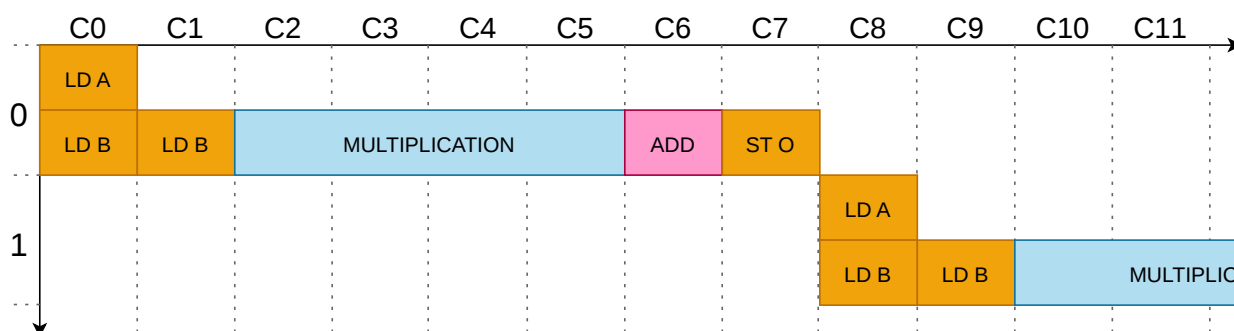
Question 2.1. With the base code only (i.e., no optimization). Fill the scheduling template with only 2 iterations and the first 12 cycles *for this question only*.

Answer.

Code Modification:

None, base code.

Scheduling Template:



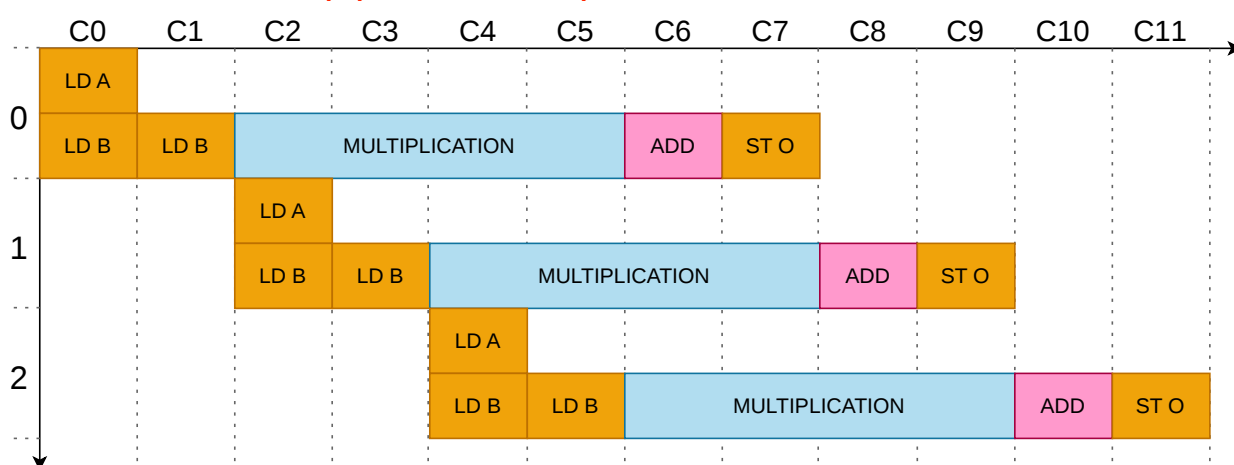
Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				(a)	(b)	(o)
8	$8 \times 1024 = 8192$	8	1	1	1 read	1 read	1 write

Question 2.2. Apply loop pipelining.**Answer.****Code Modification:**

```

1 void simpleDesign(int a[1024], int b[2048], int o[1024]) {
2     loop: for(int i = 0; i < 1024; ++i){
3         #pragma HLS PIPELINE II=2 style=stp
4         o[i] = a[i] + b[2*i] * b[2*i+1];
5     }
6 }

```

Scheduling Template: **pipelined multiplier**

Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				(a)	(b)	(o)
8	$2 \times 1024 + 6 = 2054$	2	1	1	1 read	1 read	1 write

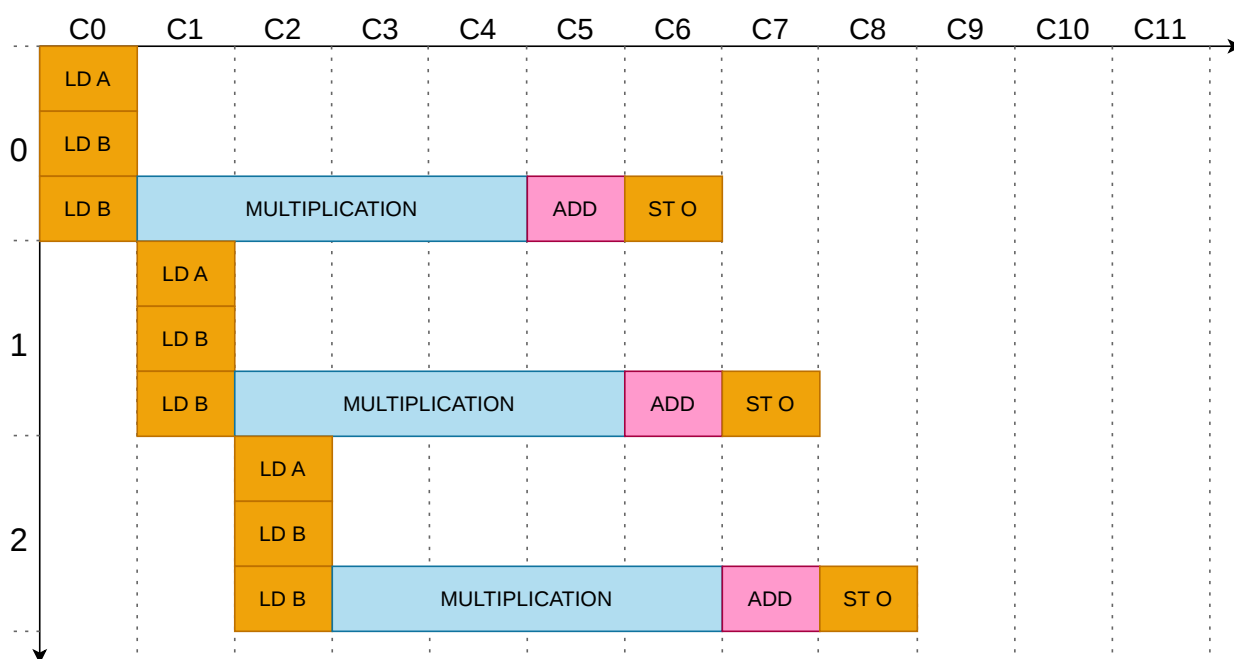
- Depending on the scheduling algorithm, LD A could also be scheduled one cycle earlier from iteration 1 (e.g., $C2 \rightarrow C1$, $C4 \rightarrow C3$, ...).
- rewind is not included in the pragma since there is only a single loop in the function.
- II=2 is chosen because each iteration needs two reads from array b ($b[2*i]$ and $b[2*i+1]$), while b maps to one single-port BRAM in this question. Therefore, at least two cycles are needed to serve these two reads, so II cannot be lower than 2.
- style=stp is the simplest model where the pipeline stalls if the data is not available. Since there is no stream of data, this model is the most relevant since it consumes the least resources.

Question 2.3. Apply array partitioning.**Answer.****Code Modification:**

```

1 void simpleDesign(int a[1024], int b[2048], int o[1024]) {
2     #pragma HLS array_partition variable=b type=cyclic factor=2 dim=1
3     loop: for(int i = 0; i < 1024; ++i){
4         #pragma HLS PIPELINE II=1 style=stp
5         o[i] = a[i] + b[2*i] * b[2*i+1];
6     }
7 }

```

Scheduling Template:

Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				(a)	(b)	(o)
7	$1 \times 1024 + 6 = 1030$	1	1	1	1 read	2 read	1 write

Question 2.4. Apply loop unrolling. Due to area constraints, assume you can only instantiate two multipliers in the design and a maximum of 4 read/write ports per array.

Answer.

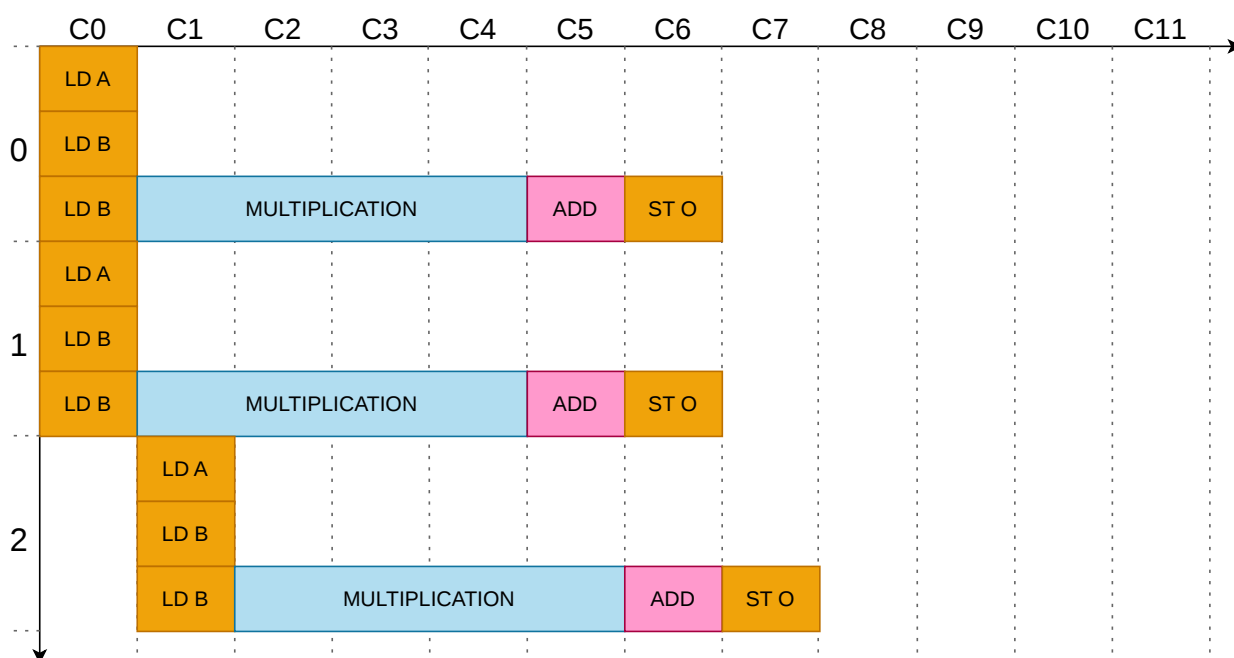
Code Modification:

```

1 void simpleDesign(int a[1024], int b[2048], int o[1024]) {
2     #pragma HLS array_partition variable=a type=cyclic factor=2 dim=1
3     #pragma HLS array_partition variable=b type=cyclic factor=4 dim=1
4     #pragma HLS array_partition variable=o type=cyclic factor=2 dim=1
5
6     #pragma HLS unroll factor=2 skip_exit_check
7     loop: for(int i = 0; i < 1024; ++i){
8         #pragma HLS PIPELINE II=1 style=stp
9         o[i] = a[i] + b[2*i] * b[2*i+1];
10    }
11 }

```

Scheduling Template:



Performance			Resources Required				
Latency (cycles)		II	Adders	Multipliers	Ports		
Per iter.	Total				(a)	(b)	(o)
7	$\lceil 1/2 \times 1024 \rceil + 6 = 518$	1	2	2	2 read	4 read	2 write

- We also partition a and o because unrolling by a factor of 2 makes the hardware execute two loop iterations in parallel, so it needs two concurrent accesses to the input array a and two concurrent writes to the output array o.
- We use `cyclic` because the accesses are consecutive across unrolled iterations. Cyclic partitioning spreads neighboring elements across different banks, which matches this access pattern and lets the two iterations run in parallel without port conflicts.
- We use `skip_exit_check` because the loop trip count is known and is a multiple of the unrolling factor, so the extra hardware that checks for a partial leftover iteration is unnecessary.

In a more realistic or smarter static compiler, cache analysis could exploit deterministic spatial locality, so assuming every access misses is more conservative than the true worst case for this simple sequential loop. However, in this lab, Static HLS is modeled as using a fixed worst-case memory latency for every access, while Dynamic HLS is allowed to benefit from actual cache hits at runtime.

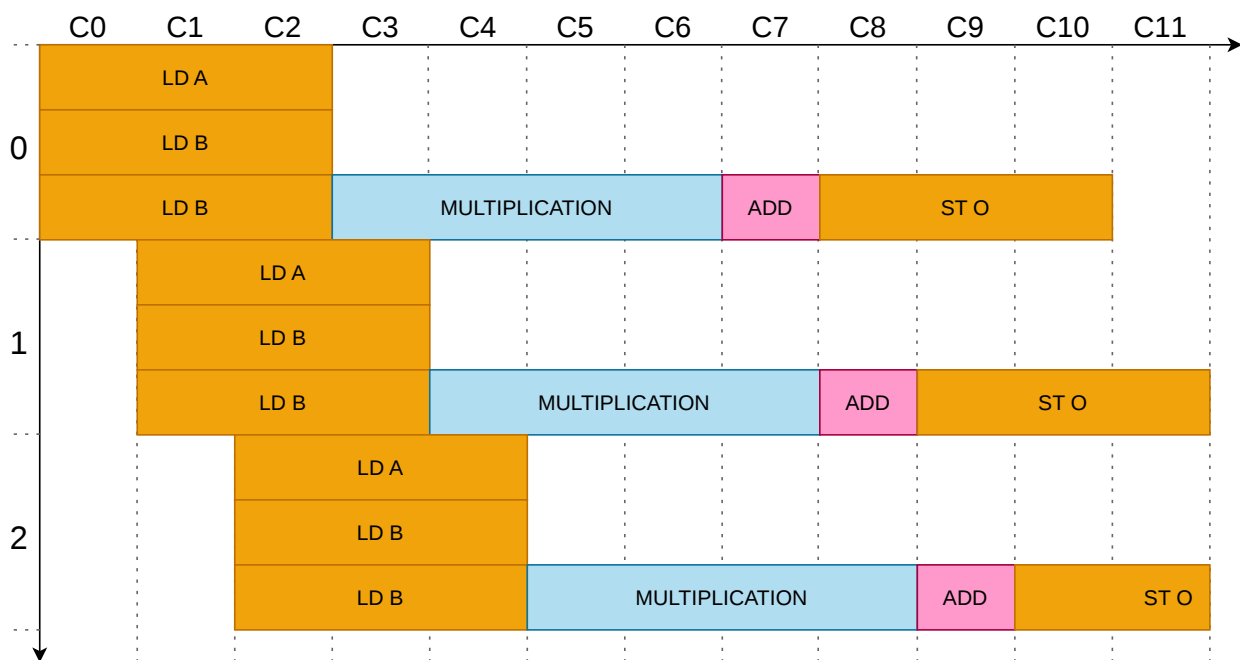
3 Dynamic HLS

Consider now a latency of 3 cycles on memory operations. To circumvent this additional latency, the architecture implements a write-back data cache per array, with a block size of 4 integers. A cache hit takes 1 cycle while a cache miss can be resolved in 3 cycles. Note that the cache is pipelined in nature, i.e., can accept a request every single cycle, and does **not** implement a hardware prefetcher.

Question 3.1. Assuming the data cache is initially empty, produce the scheduling from the Question 2.3, i.e., with pipelining and array partitioning but no unrolling, considering this new constraint in a Static HLS environment.

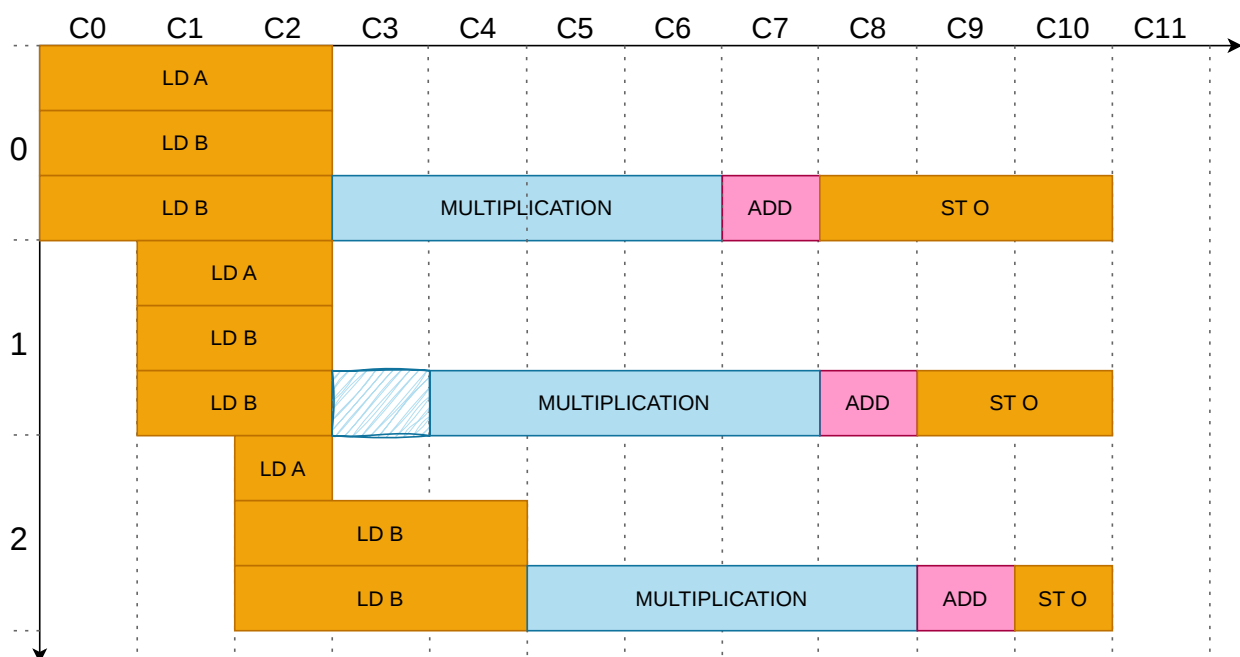
Hint: Schedule it for the worst-case scenario, i.e., assume that every access produces a cache miss.

Answer. Scheduling Template:



Question 3.2. Would Dynamic HLS help? Produce once again the scheduling from the previous example, this time considering dynamic HLS.

Answer. Scheduling Template:



- In cycle 3, iteration 1, the multiplication is ready to be started. However, since there is only 1 multiplier, the pipeline is stalled and waits the next cycle to be scheduled.